



Centro Universitário de Excelência
Sistemas de Informação

MecâniQA — Web API Fundacional

Gestão de Catálogo de Autopeças e Serviços em Memória

Equipe: São Luiz

Autor: Hellen Verena da Conceição Magalhães
Italo Yan Mendes da Silva

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é modernizar a gestão da startup MecâniQA (Feira de Santana) com uma API RESTful em Java/Spring Boot sob rigoroso isolamento arquitetural em memória.

1. Contexto & Desafio:

O problema da MecâniQA e as restrições da OAT 1.

2. Modelagem UML:

Diagrama de Classes, visibilidade, tipagem e tipos enumerados.

3. Singleton Manual:

Gestão de estado em memória RAM e ciclo de vida na JVM.

4. Camada RESTful:

Roteamento, blindagem com DTOs e semântica com **ResponseEntity**.

5. Testes de Integração:

Validação automatizada das 8 User Stories no Postman.

1. Como a Requisição percorre a API:



Entrada

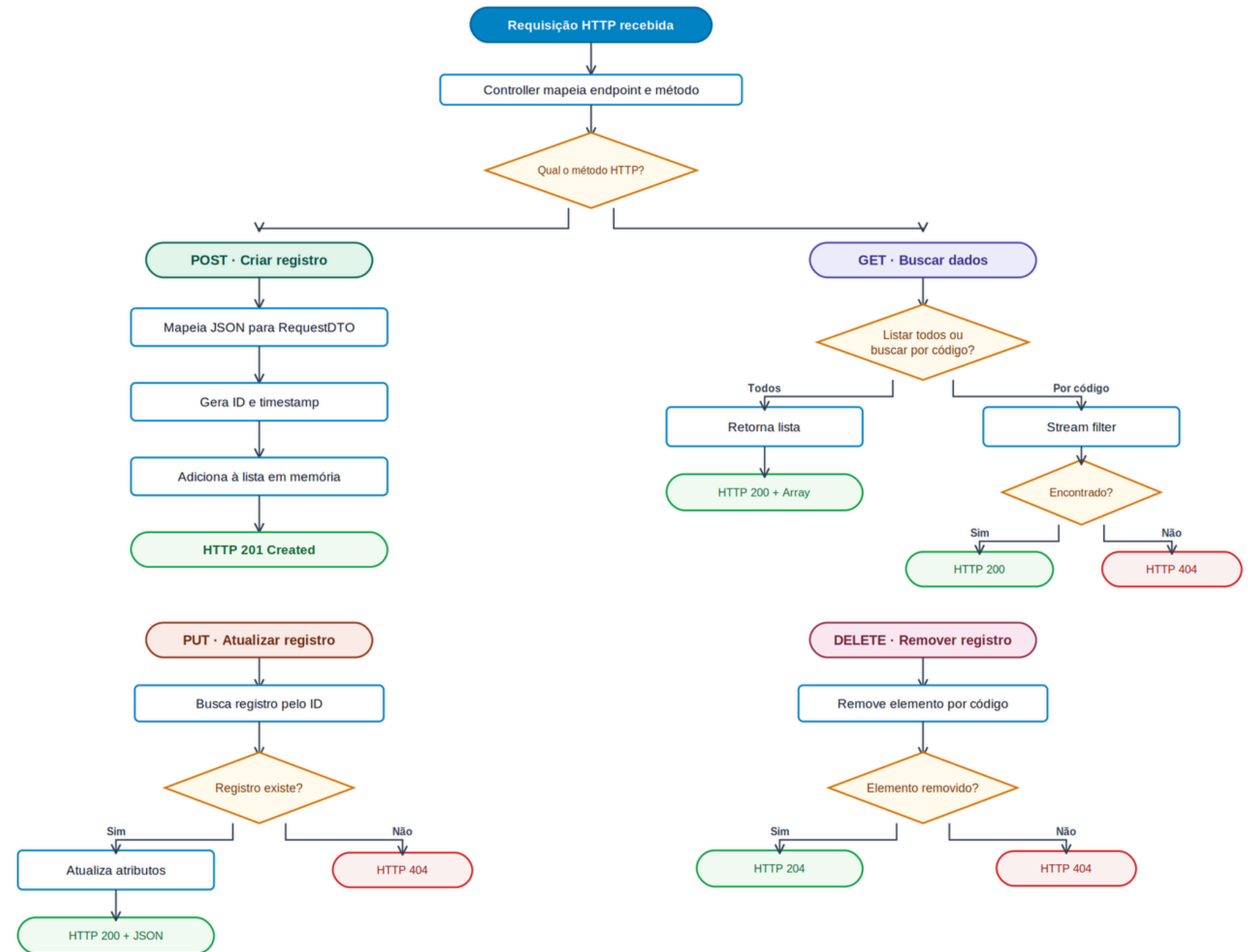
Controller recebe a requisição HTTP e o DTO transporta dados de POST e PUT.

Processamento

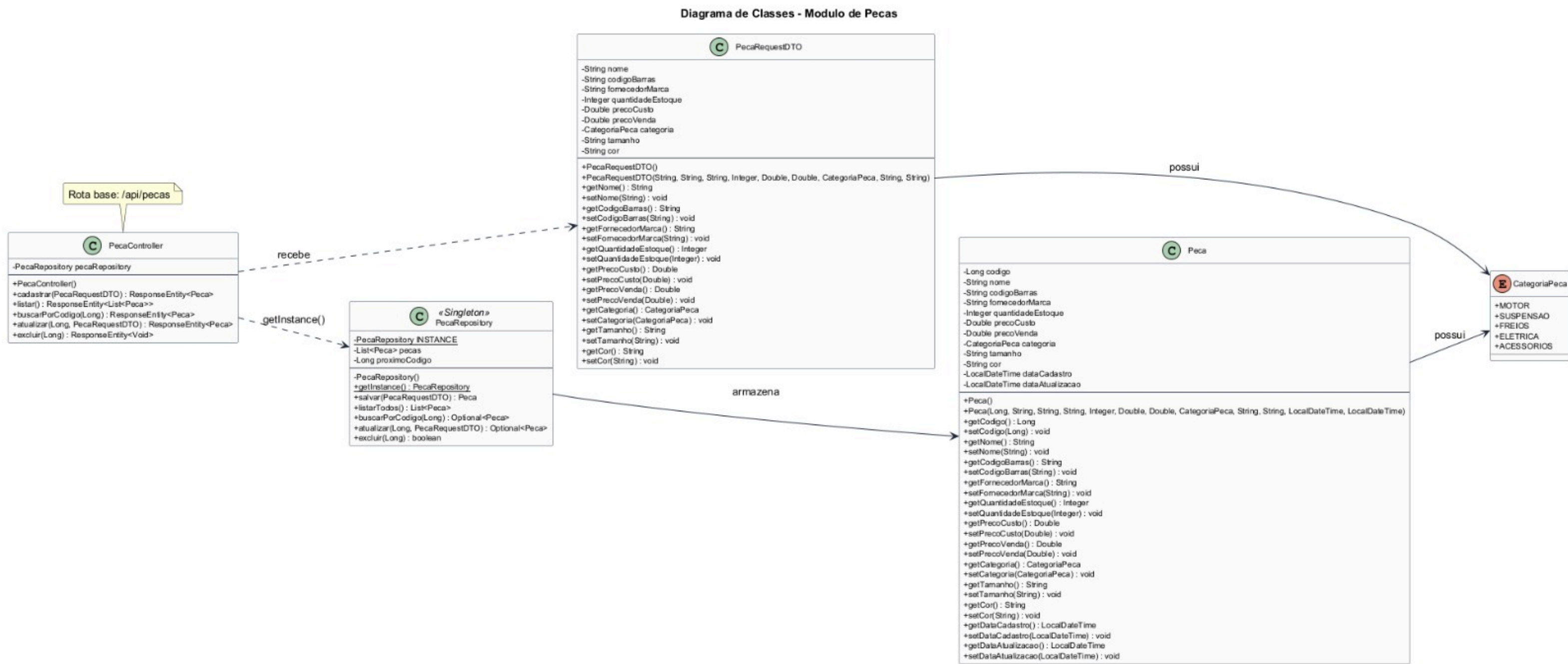
Repository Singleton executa o CRUD em memória com proteção nas escritas.

Resposta

A Api retorna 201, 200, 204 ou 404 conforme o resultado.

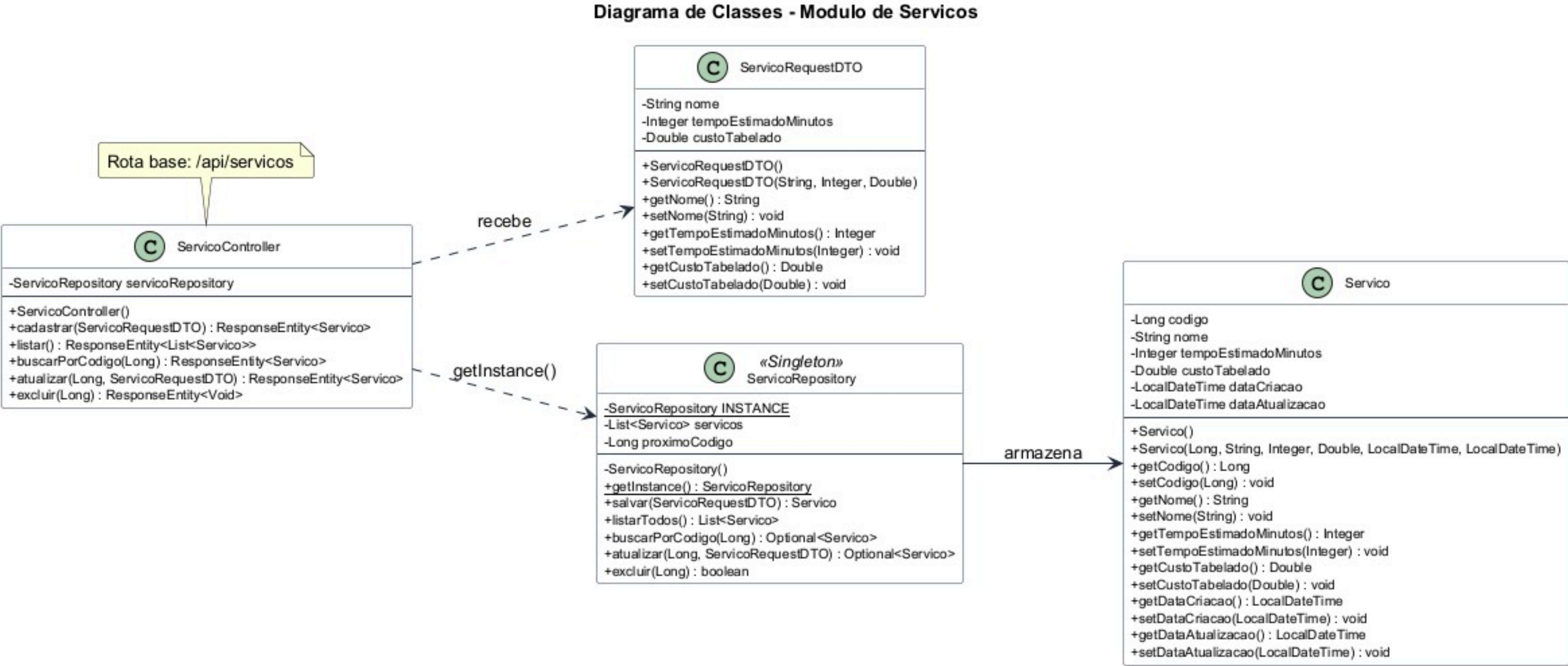


Modelagem Estrutural – Módulo de Peças & Enum (CategoriaPeca)



US01 a US04: Encapsulamento, Type Safety e Blindagem com DTO

Modelagem Estrutural – Módulo de Serviços & Singleton



US05 a US08: Padrão Singleton Manual e Persistência na Memória JVM

Padrão Singleton Manual & Gestão de Estado na JVM



Uma referência estática reutiliza o mesmo repositório enquanto a JVM está ativa.



```
1  public class PecaRepository {
2      private static PecaRepository INSTANCE;
3      private final List<Peca> pecas = new ArrayList<>();
4      private Long proximoCodigo = 1L;
5
6      private PecaRepository() {}
7
8      public static PecaRepository getInstance() {
9          if (INSTANCE == null) {
10             INSTANCE = new PecaRepository();
11         }
12
13         return INSTANCE;
14     }
15 }
```

1. Criação controlada

O construtor private impede new fora da classe.

2. Estado compartilhado

INSTANCE referência o repositório. O objeto, a ArrayList e o controlador permanecem na Heap.

3. Ciclo de vida

Os Controllers reutilizam getInstance(). Os dados desaparecem quando a JVM reinicia.

Camada RESTful — Roteamento, DTOs & Semântica HTTP



Roteamento define a ação, DTO controla a entrada e ResponseEntity comunica o resultado.



```
1 @PostMapping
2 public ResponseEntity<Peca> cadastrar(
3     @RequestBody PecaRequestDTO dto) {
4     Peca peca = pecaRepository.salvar(dto);
5     return ResponseEntity.status(HttpStatus.CREATED).body(peca);
6 }
```

- `@PostMapping` → associa o método ao verbo POST
- `PecaRequestDTO` → recebe somente os dados de entrada
- `ResponseEntity` → devolve 201 Created com o objeto

POST

/api/pecas

201 Created (Novo Objeto)



GET

/api/pecas/{id}

200 OK / 404 Not Found



PUT

/api/pecas/{id}

200 OK (Atualizado)



DELETE

/api/pecas/{id}

204 No Content



Testes de Integração

Validação automatizada das 8 User Stories (US01 a US08) e conformidade de status HTTP no Postman



US01 POST /api/pecas → 201 Created

US02 GET /api/pecas → 200 OK

US03 GET /api/pecas/{id} → 200 OK

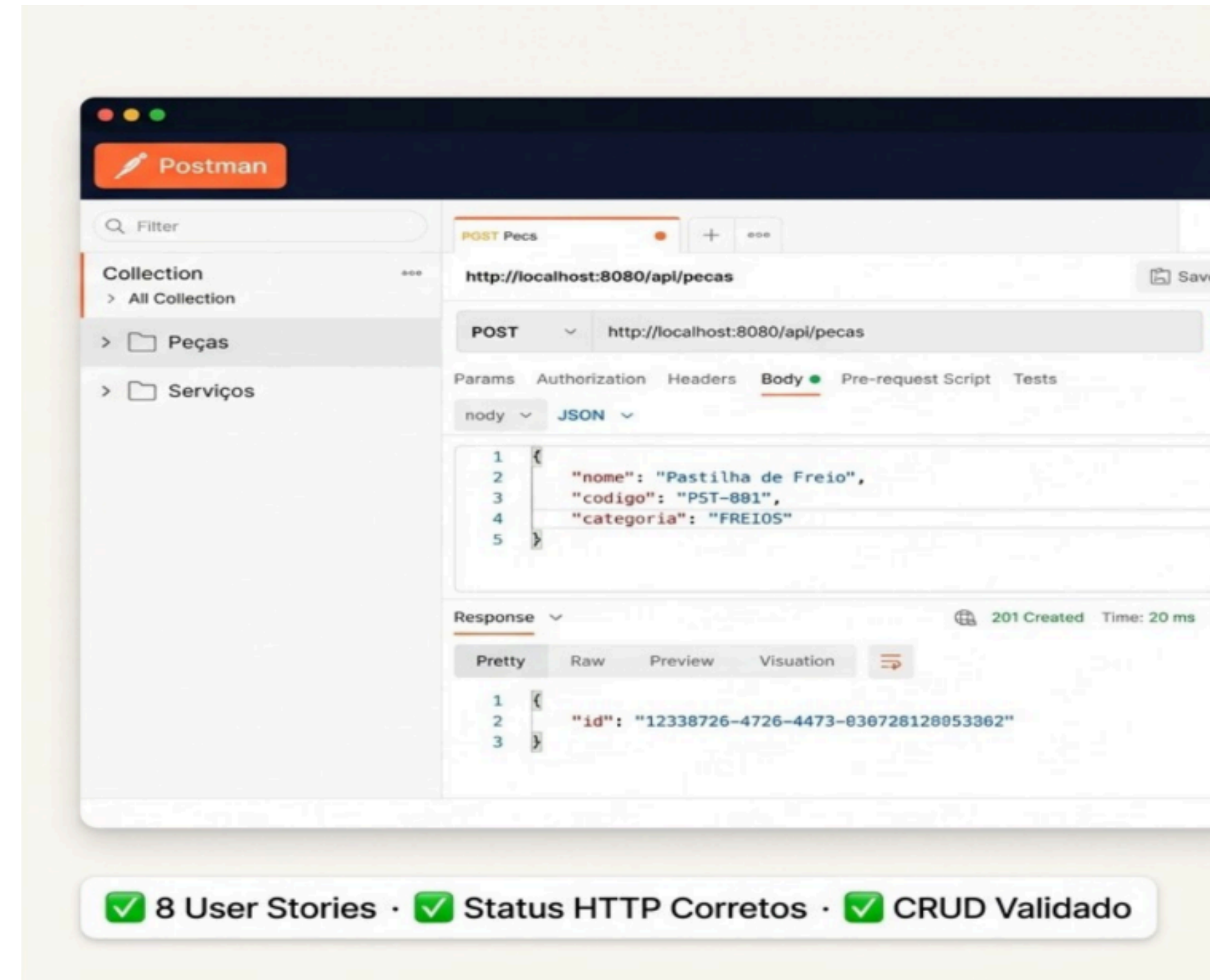
US04 PUT /api/pecas/{id} → 200 OK

US05 DELETE /api/pecas/{id} → 204

US06 POST /api/servicos → 201 Created

US07 GET /api/servicos → 200 OK

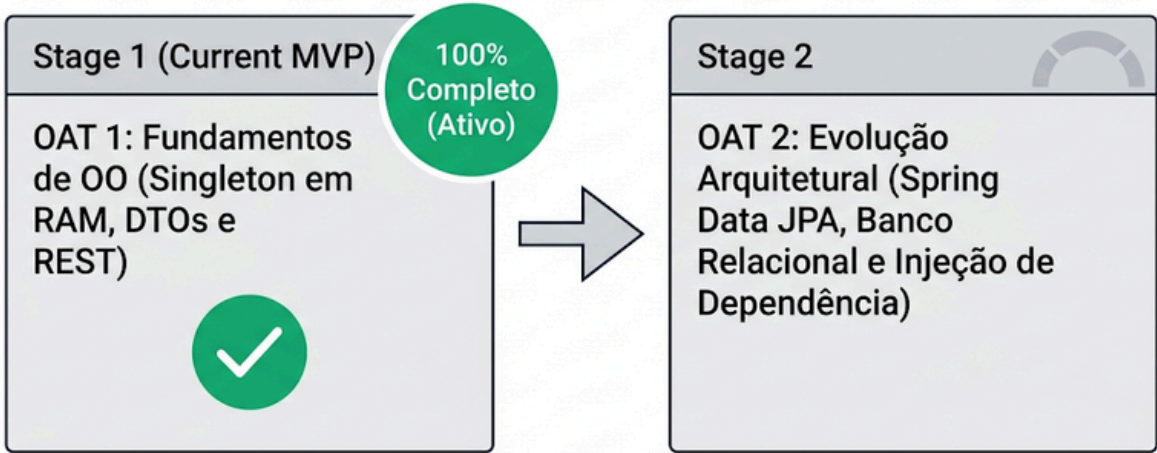
US08 DELETE /api/servicos/{id} → 204



Considerações Finais — Conclusão do MVP & Próximos Passos



Sprint Concluída com Sucesso



**CRUD Completo**
8 User Stories entregues — Peças e Serviços

**Singleton Manual**
Estado em memória RAM — sem banco de dados

**API RESTful**
/api/pecas · /api/servicos · HTTP semântico

**Type Safety com Enum**
CategoriaPeca: MOTOR · FREIOS · ELETRICA...

Referências



- POSTMAN. Postman API Platform Documentation: collection runner and test scripts. 2026. Disponível em: Referências do projeto. Acesso em: 29 ago. 2026.
- VMWARE. Spring Boot Reference Documentation: building RESTful web services. Versão 3.x. 2026. Disponível em: Referências do projeto. Acesso em: 29 ago. 2026.
- GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.
- OBJECT MANAGEMENT GROUP (OMG). Unified Modeling Language (OMG UML): Version 2.5.1. Needham: OMG, 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1>. Acesso em: 29 ago. 2026.